

Programmeringsparadigmer 2025

Kursusgang 11: Monader

Opgaver til løsning og diskussion

Hans Hyttel

18. november 2025

Opgaver, som vi helt sikkert vil tale om

1. (**10 minutter**) To influencere havde en ophedet diskussion om List-monaden. Influencer 1 præsenterede en ny funktion kaldet `fourfirst` :

```
fourfirst xs = do
    x <- xs
    return (4, x)
```

”Denne funktion tager en liste og giver os et par $(4, x)$, hvor x er det første element i listen,” sagde den første influencer.

”Du tager fejl,” sagde Influencer 2. ”Koden tildeler xs til variabelen x , og vi får et par, hvor den første komponent er 4 og den anden komponent er x , som er en liste, der også er kendt som xs . ”

Forklar, ved hjælp af definitionen af List-monaden *men uden at eksekvere koden*, hvad denne funktion egentlig gør, og hvorfor.

2. (**25 minutter**) Her er et stykke Haskell-kode.

```
data W x = Bingo x deriving Show
instance Functor W where
    fmap f (Bingo x) = Bingo (f x)
instance Monad W where
    return x = Bingo x
    Bingo x >>= f = f x
```

- a. For at dette giver mening, mangler der en definition af `W` som en applikativ funktor. Skriv en sådan definition.
- b. Et stort tøjfirma sponsorerer et populært tv-show og bad stjernen om at definere en funktion `wrapadd :: Num b => b -> W b -> W b`, der opfylder, at

$$\text{wrapadd (Bingo } x\text{)}(\text{Bingo } y\text{)} = \text{Bingo } (x+y)$$

og udnytter, at `W` er en monade. Definitionen skulle bruges i reklamer på sociale medier for en serie nye jakker. TV-stjernen kom med følgende definition:

```
wrapadd x (Bingo y) = do
    Bingo (x+y)
```

Tøjfirmaet klagede dog over, at denne definition var klodset — den brugte ikke monader på en fornuftig måde. TV-stjernen blev bedt om at revidere definitionen. Hvad er galt med den?

3. (**30 minutter**) I kursusgang 7 så vi på en Haskell-datatype `Aexp` for aritmetiske udtryk med addition, multiplikation, tal og variable. Dannelsesreglerne er:

$$E ::= n \mid x \mid E_1 + E_2 \mid E_1 \cdot E_2$$

Vi antager, at variable x er strenge, og at tal n er heltal.

Vi blev bedt om at definere en funktion `eval`, som, når den får en term af typen `Aexp` og en værdi-tildeling `ass` af variable til tal, vil beregne værdien af udtrykket. Som eksempel, hvis vi har

```

data Aexp = Var String | Num Integer | Plus Aexp Aexp | Mult Aexp Aexp
look ass x = head [ v | (y,v) <- ass, y == x ]
eval (Var x) ass = look ass x
eval (Num n) ass = n
eval (Plus a1 a2) ass = v1 + v2
                        where
                        v1 = eval a1 ass
                        v2 = eval a2 ass
eval (Mult a1 a2) ass = v1 * v2
                        where
                        v1 = eval a1 ass
                        v2 = eval a2 ass

— En tildeling (eksempel)

myass = [( "x" ,3) , ( "y" ,4) ]

```

Figure 1: Den oprindelige løsning, der definerer en evaluator

tildelingen $[x \mapsto 3, y \mapsto 4]$, skal `eval` fortælle os, at værdien af $2 \cdot x + y$ er 10. Figur 1 viser koden for en løsning på denne opgave.

Ovenstående definition kan dog ikke håndtere tilfælde, hvor vi forsøger at evaluere et udtryk, der indeholder variable, hvis værdi ikke er givet i den tildeling, vi angiver.

Som eksempel, hvis vi forsøger at evaluere udtrykket $2 + z$ med tildelingen $[x \mapsto 3, y \mapsto 4]$, vil der opstå en fejl.

Din opgave er nu at *bruge Maybe-monaden* til at definere en ny evaluator, `eval'`, således at `eval' e ass` returnerer `Just v`, hvis `e` evaluerer til værdien `v`, og returnerer `Nothing`, hvis `e` indeholder variable, der ikke er bundet i `ass`.

Find mindst fire gode testtilfælde.

Flere opgaver at løse i dit eget tempo

- a) Definer en `foldM`-funktion, hvis type skal være:

```
foldM :: Monad m => (t1 -> t2 -> m t2) -> [t1] -> t2 -> m t2
```

Idéen er, at funktionen virker som `foldl`, men folder over en monade. Her er et eksempel, der viser, hvad der sker, hvis vi folder over `IO`-monaden. Hvis vi lader:

```
dingo x = do
    putStrLn (show x)
    return x
```

så bør vi se følgende adfærd:

```
*Main> foldM (\x y -> (dingo (x+y))) [1,2,3,4] 0
1
3
6
10
```

- b) Betragt træer, hvis elementer er værdier af en type i typeklassen `Ord`. Typen `Tree a` er defineret ved:

```
data Tree a = Leaf a | Node (Tree a) (Tree a)
```

Brug en monade til at skrive en funktion `minorder`, der tager et sådant træ og tjekker, om tallene i strukturen er i ikke-aftagende orden, når de læses fra venstre mod højre. Hvis det er tilfældet, skal funktionen returnere `Just k`, hvor `k` er det mindste tal i træet, ellers skal den give os `Nothing`.

Træet `mytree1` vist i figur 2 er ordnet, så `minorder mytree1` skal returnere `Just "aha"`, men træet i figur 3 er ikke ordnet, så `minorder mytree2` skal returnere `Nothing`.

Definer først en anden funktion `minmax`, der finder det mindste og det største element i et træ under antagelse af, at træet er ordnet. Brug derefter `minmax` til at definere `minorder`.

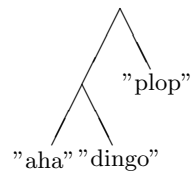


Figure 2: Det ordnede træ mytree1

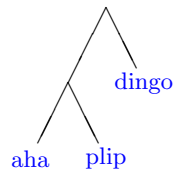


Figure 3: Det uordnede træ mytree2

Hint: Find først nogle gode testtilfælde. Find derefter ud af, hvilken monade du skal bruge. ***Advarsel!*** Brug kun monadisk notation!